Hospital ER Patient Management System (Priority Queue) — Report Template

Date: January 12, 2026

Course: Programming with Linear Structures

University: Universidad Europea de Madrid

1. Group members

- <Name 1>
- <Name 2>
- <Name 3>

2. Problem statement

Implement a patient management system for a hospital emergency room using a dynamically implemented priority queue (linked list). Patients have a priority level from 1 (highest urgency) to 5 (lowest urgency). Patients with the same priority are treated in order of arrival.

3. Data structures

• Patient: name (string), reason (string), priority (1–5), arrival counter (FIFO tie-breaker).
• Node: linked-list node storing one Patient and a pointer to next node.
• PriorityQueue: pointer to front node and a counter used to assign arrival order on enqueue.

4. Algorithms

• Enqueue: insert patient into the linked list so that lower priority numbers are closer to the front. For equal priority, keep FIFO by inserting after existing nodes of the same priority.
• Dequeue: remove the front node only. Return the removed Patient data to the caller (GUI), which prints it in the log.
• Peek: return the front patient without removing it (used for “announce next patient”).
• Print: traverse from front to back and format each patient into the GUI log.

5. GUI (GTK) design and use

• The GUI is built entirely in main.c using GTK.
• A menu bar triggers queue operations: Admit patient, Care next patient (dequeue), Announce next patient (peek), Print full queue, Check if empty, Quit.
• All messages and results are written to a scrollable text log (GtkTextView).
• Admit patient opens a dialog that collects Name, Reason, and Priority (1–5).

6. Results and discussion

Describe test scenarios you executed in the GUI: mixed priorities, repeated same-priority admissions, dequeue order validation, empty queue behavior, and memory cleanup on exit.

Add screenshots of the GUI log here if required by your professor.